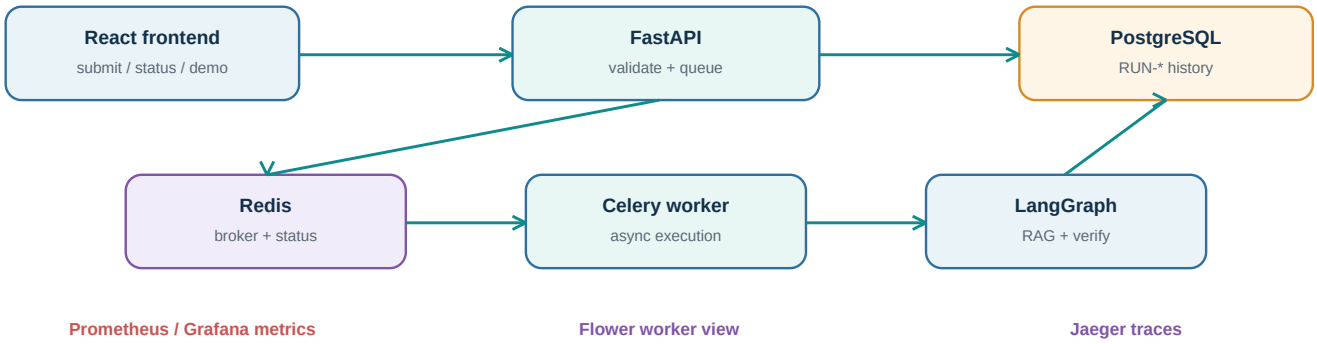
Health Insurance Pre-Authorization Desk

Compact implementation report: architecture, implemented features, and how the working demo operates



Implemented system

Area	What was implemented
Frontend	React Submit, Track Status, Live Records, Cataract Demo, result and feedback views.
Backend	FastAPI request contract, validation, Run ID creation, async submission, polling, feedback, health, and metrics endpoints.
Agents	LangGraph classifier, verifier, retrieval tools, self-verification, retries, timeouts, and escalation for unsupported evidence.
Async path	Redis broker/status layer, Celery worker, queue routing, task retries, and Flower task monitoring.
Storage	PostgreSQL RUN-* payload/result history and feedback when available; Redis remains the fast live layer and fallback.
Demo data	20 seeded DEMO-* records plus 50 manual-only synthetic samples.
Observability	Prometheus, Grafana, Jaeger, structured Docker logs, latency, queue, outcome, and error metrics.

Request flow

- User submits synthetic patient, policy, hospital, and medical fields from React.
- FastAPI validates schema, ranges, categories, dates, age consistency, and free-text fields.
- A RUN-* ID is returned immediately; PostgreSQL receives the pending row when available.
- Redis/Celery sends work to the worker; LangGraph retrieves policy clauses and verifies the route.
- The worker writes recommendation, citations, steps, token usage, timing, prompt version, and final state.
- The frontend polls the status endpoint and displays the cited recommendation and escalation path.

The same flow supports a Mumbai hospital or any other valid hospital payload: it receives a live RUN-* result without being added to the seeded Live Records dashboard.

What we implemented and how

FastAPI and validation

Pydantic models enforce patient, policy, hospital, and medical contracts before an agent starts. This covers identifier patterns, age/date consistency, policy dates and coverage, ICD-10 format, admission categories, costs, discharge dates, and free-text length. Invalid or suspicious input is rejected or quarantined by the guardrail path.

Agent reasoning

Stage	Implementation
Classifier	Identifies admission type and extracts clinical/policy entities for the coverage decision.
Retriever	Searches the FAISS policy corpus and can look up clauses and request fields.
Verifier	Checks coverage, waiting periods, exclusions, and supporting passages away from the main cover clause.
Self-verification	Blocks unsupported results, retries evidence gathering, and escalates when evidence is insufficient.

Frontend and demo behavior

The original 20 DEMO-* records are shown in Live Records. The extra 50 samples are entered manually from MANUAL_DEMO_GUIDE.txt, receive RUN-* IDs, and appear on their own status pages rather than in the dashboard. The samples cover emergency, planned, day-care, network, non-network, approval, query, and medical-review routes. The cataract case demonstrates the separate 24-month waiting-period citation and medical-review route.

Durable run history

The preauth_runs row is created before queueing when PostgreSQL is available. The worker updates pending, processing, completed, or failed state and stores the request payload, route, recommendation, escalation reason, citations, agent steps, token usage, prompt version, timing, and completion time. Feedback is stored in preauth_feedback. If PostgreSQL is unavailable, the request still runs through Redis/Celery and the status endpoint can use the Redis result for its configured cache lifetime.

Monitoring and performance

Tool	Working use
Prometheus / Grafana	Throughput, p50/p95 latency, queue depth, error rate, outcomes, tool-call rate, and security events.
Flower	Celery workers, active tasks, and task states.
Jaeger	OpenTelemetry traces for backend and worker services.
Docker logs	Structured request, worker, agent, tool, and error logs on stdout.
Locust	Load test with RPS, p50, p95, and error-rate reporting.

Run the demonstration

```
cd C:\Users\saira\OneDrive\Desktop\MLSDA_mini
docker compose up -d --build
docker compose run --rm corpus-ingest
docker compose exec backend python /app/scripts/seed_demo_data.py
Start-Process http://localhost:3000
```

Open: application http://localhost:3000 | Swagger http://localhost:8000/api/docs | Flower http://localhost:5555 | Grafana http://localhost:3001 | Prometheus http://localhost:9090 | Jaeger http://localhost:16686

Completed work

- Added and validated 50 manual synthetic request payloads without expanding the dashboard seed.
- Connected live RUN-* persistence and PostgreSQL fallback status lookup.
- Added Flower dependency and fixed the Grafana Prometheus datasource UID.
- Expanded README and manual demo guide with setup, run, stop, metrics, and screenshot guidance.
- Kept Track Status available for revisiting RUN-* requests while keeping the dashboard limited to DEMO-* records.